

1. Relational Model and its Components (10 Points)

1. **Relational Model** – It is a database model that stores data in the form of tables (relations).
2. Tables consist of rows and columns to represent data and relationships.
3. **Relation (Table)** – A collection of data organized in rows and columns.
4. Example: `Student(ID, Name, Age)`
5. **Attribute (Column)** – Represents a property of an entity.
6. Example: ID, Name, Age are attributes of Student table.
7. **Tuple (Row)** – A single record in a table.
8. Example: (1, 'Tanmay', 20)
9. Each table has a **schema** (structure) and **instance** (data).
10. The relational model is simple, flexible, and easy to use for managing data.

2. Attributes and Domains (10 Points)

1. **Attribute** – A column in a table that represents a property of an entity.
2. It defines what type of data is stored in that column.
3. Example: In `Student(ID, Name, Age)` → ID, Name, Age are attributes.
4. Each attribute has a **name and data type**.
5. **Domain** – A set of valid values that an attribute can take.
6. It restricts the type and range of values.
7. Example: Age domain → values from 1 to 100
8. Example: Gender domain → 'Male', 'Female'
9. Domain helps in maintaining data accuracy and consistency.
10. So, attributes define columns and domains define valid values for those columns.

♦ Rule 4 (Active Catalog) – Meaning

“Metadata stored as tables” means:

👉 The database stores information about itself in table form, just like normal data.

♦ First understand “Metadata”

Metadata = data about data

Examples:

- Table names
 - Column names
 - Data types
 - Constraints (like primary key)
-

♦ What Rule 4 says

Instead of storing this metadata in some hidden or special format,

👉 the DBMS stores it in **regular tables** (called **system tables**)



Rule 4 states that all metadata (like table structure, columns, etc.) must be stored in the database as tables and can be accessed using SQL.

1. **Codd's Rules** – Set of rules to define a proper relational database system.
2. **Rule 0 (Foundation Rule)** – DBMS must be relational.
Eg: Data stored in tables only.
3. **Rule 1 (Information Rule)** – All data stored in rows & columns.
Eg: Student(ID, Name)
4. **Rule 2 (Guaranteed Access Rule)** – Data accessed using table + key + column.
Eg: `SELECT Name FROM student WHERE ID = 1;`
5. **Rule 3 (NULL Values Rule)** – Supports NULL for missing data.
Eg: Age = NULL
6. **Rule 4 (Active Catalog)** – Metadata stored as tables.
Eg: Table info stored in system tables.
7. **Rule 5 (Comprehensive Language)** – One language (SQL) for all operations.
Eg: SELECT, INSERT, UPDATE
8. **Rule 6 (View Updating Rule)** – Some views can be updated.
Eg: `UPDATE view_name SET Name='A';`
9. **Rule 7 (High-Level Operations)** – Works on mul  rows at once.
Eg: `UPDATE student SET marks=50;`
10. **Rule 8 (Physical Data Independence)** – Storage changes don't affect user.

4. Integrity Constraints and Types (10 Points)

1. **Integrity Constraints** – Rules that ensure accuracy and consistency of data in database.
2. They prevent invalid data entry and maintain data correctness.
3. **Domain Constraint** – Ensures values are within valid range/type.
4. *Example:* Age must be between 1–100
5. **Entity Integrity Constraint** – Primary key cannot be NULL or duplicate.
6. *Example:* Student ID must be unique and not null
7. **Referential Integrity Constraint** – Foreign key must match primary key of another table.
8. *Example:* Dept_ID in student must exist in department table
9. **Enterprise Constraint** – Business rules defined by organization.
10. *Example:* A student can enroll in max 5 courses

7. Rule 5 (Comprehensive Language) – One language (SQL) for all operations.

Eg: SELECT, INSERT, UPDATE

8. Rule 6 (View Updating Rule) – Some views can be updated.

Eg: UPDATE view_name SET Name='A';

9. Rule 7 (High-Level Operations) – Works on multiple rows at once.

Eg: UPDATE student SET marks=50;

10. Rule 8 (Physical Data Independence) – Storage changes don't affect user.

Eg: Changing file storage doesn't change query results.

4. Integrity Constraints and Types (10 Points)

1. **Integrity Constraints** – Rules that ensure accuracy and consistency of data in database.
2. They prevent invalid data entry and maintain data correctness.
3. **Domain Constraint** – Ensures values are within valid range/type.
4. *Example:* Age must be between 1–100
5. **Entity Integrity Constraint** – Primary key cannot be NULL or duplicate.
6. *Example:* Student ID must be unique and not null
7. **Referential Integrity Constraint** – Foreign key must match primary key of another table.
8. *Example:* Dept_ID in student must exist in department table
9. **Enterprise Constraint** – Business rules defined by organization.
10. *Example:* A student can enroll in max 5 courses

5. Entity Integrity and Referential Integrity (10 Points)

1. **Integrity Constraints** ensure data is accurate and consistent in database.
2. **Entity Integrity** – Primary key must be **unique and not NULL**.
3. It ensures each record is uniquely identified.
4. *Example:* In `Student(ID, Name)` → ID cannot be NULL or duplicate
5. If ID is NULL → record cannot be identified
6. **Referential Integrity** – Foreign key must match primary key of another table.
7. It maintains relationship between tables.
8. *Example:* `Student(Dept_ID)` refers to `Department(Dept_ID)`
9. If Dept_ID does not exist in Department table → error
10. These constraints maintain correctness and relationship of data in database

6. Domain Constraint (10 Points)

1. **Domain Constraint** – It ensures that an attribute contains only valid values.
2. It defines the **type, range, and format** of data in a column.
3. It prevents invalid data entry in the database.
4. Domain is a set of allowed values for an attribute.
5. It is the most basic type of integrity constraint.
6. **Example:** Age should be between 1 and 100
7. **Example:** Gender should be 'Male' or 'Female'
8. Can be applied using constraints like `NOT NULL`, `CHECK`, `DEFAULT`
9. **Example:** `Age INT CHECK (Age > 0)`
10. Domain constraint helps maintain data accuracy and consistency

7. Functional Dependency (10 Points)

1. **Functional Dependency (FD)** – Relationship where one attribute determines another.
2. It is written as $A \rightarrow B$ (A determines B).
3. *Example:* If ID determines Name $\rightarrow ID \rightarrow Name$
4. **Full Functional Dependency** – Attribute depends on whole key.
5. *Example:* $(ID, Course) \rightarrow Grade$ (depends on both)
6. **Partial Dependency** – Attribute depends on part of key.
7. *Example:* $(ID, Course) \rightarrow Name$ (depends only on ID)
8. **Transitive Dependency** – Dependency through another attribute.
9. *Example:* $ID \rightarrow Dept_ID$ and $Dept_ID \rightarrow Dept_Name \Rightarrow ID \rightarrow Dept_Name$
10. FD helps in normalization and reducing redundancy

◆ What is Transitive Dependency?

👉 It means one attribute depends on another through a third attribute

★ In short:

$A \rightarrow B$ and $B \rightarrow C \Rightarrow A \rightarrow C$ (indirectly)

◆ Simple Real-Life Example

Think like this:

- $\text{Student_ID} \rightarrow \text{Dept_ID}$
- $\text{Dept_ID} \rightarrow \text{Dept_Name}$

👉 So automatically:

$\text{Student_ID} \rightarrow \text{Dept_Name}$

But...

- 👉 Student_ID is NOT directly deciding Dept_Name
- 👉 It is happening through Dept_ID

This is called Transitive Dependency

◆ Table Example

Student_ID

Dept_ID



Dept_Name

8. Partial, Full, and Transitive Dependency (10 Points)

1. These are types of **Functional Dependencies** used in normalization.
2. **Full Functional Dependency** – Attribute depends on the whole primary key.
3. It does not depend on any subset of the key.
4. *Example:* (ID, Course) $\rightarrow$ Grade (depends on both ID and Course)
5. **Partial Dependency** – Attribute depends on part of the primary key.
6. Occurs when key is composite (multiple attributes).
7. *Example:* (ID, Course) $\rightarrow$ Name (depends only on ID)
8. **Transitive Dependency** – Attribute depends on another non-key attribute.
9. *Example:* ID $\rightarrow$ Dept_ID and Dept_ID $\rightarrow$ Dept_Name $\Rightarrow$ ID $\rightarrow$ Dept_Name
10. Removing these dependencies helps achieve normalization and reduces redundancy

9. Normalization and its Importance (10 Points)

1. **Normalization** – Process of organizing data in database to reduce redundancy and dependency.
2. It divides large tables into smaller related tables.
3. It improves structure and efficiency of database.
4. **Importance:** Reduces duplicate data (redundancy).
5. Helps maintain data consistency.
6. Prevents anomalies (insert, update, delete problems).
7. Improves data integrity and accuracy.
8. Makes database easier to manage.
9. Improves query performance.
10. Leads to better database design (1NF, 2NF, 3NF, BCNF).

10. First Normal Form (1NF) (10 Points)

1. **1NF** – A table is in First Normal Form if all values are **atomic (single value)**.
2. Each column should contain only one value (no multiple values).
3. There should be no repeating groups in a table.
4. Each record should be unique.
5. It removes multi-valued attributes.

6. Not in 1NF Example:

`Student(ID, Name, Phone)`

(1, Tanmay, 9876, 8765) ❌ (multiple phone numbers in one field)

7. Convert to 1NF:

(1, Tanmay, 9876)

(1, Tanmay, 8765)

8. Now each field contains a single value ✔️
9. Data redundancy may increase in 1NF.

10. 1NF is the first step of normalization for better database design.

🔥 Step 3: Convert to 2NF

👉 Rule of 2NF:

Remove partial dependency

So we split the table 🗣️

◆ After Conversion (2NF)

Table 1: Student

ID	Name
1	Tanmay

👉 Here: ID → Name

Table 2: Student_Course

ID	Course	Grade
1	Java	A
1	DBMS	B

12. Third Normal Form (3NF) (10 Points)

1. 3NF – A table is in 3NF if it is in 2NF and has **no transitive dependency**.
2. Transitive dependency means a non-key attribute depends on another non-key attribute.
3. In 3NF, non-key attributes depend **only on the primary key**.
4. It removes indirect dependencies.

5. Example (Not in 3NF):

Employee(ID, Dept_ID, Dept_Name)

Here, $ID \rightarrow Dept_ID$ and $Dept_ID \rightarrow Dept_Name$ ❌

6. So Dept_Name depends indirectly on ID (transitive dependency).

7. Convert to 3NF:

Table1: Employee(ID, Dept_ID)

Table2: Department(Dept_ID, Dept_Name)

8. Now no transitive dependency ✔
9. Reduces redundancy and improves data integrity. ↓
10. 3NF is widely used for proper database design.

11. Second Normal Form (2NF) (10 Points)

1. **2NF** – A table is in 2NF if it is in 1NF and has **no partial dependency**.
2. Partial dependency means a non-key attribute depends on part of a composite key.
3. In 2NF, every non-key attribute must depend on the **whole primary key**.
4. It removes partial dependency.

5. Example (Not in 2NF):

`Student(ID, Course, Name, Grade)`

Primary Key = (ID, Course)

Here, Name depends only on ID ❌

6. This is partial dependency.

7. Convert to 2NF:

Table1: `Student(ID, Name)`

Table2: `Student_Course(ID, Course, Grade)`

8. Now all attributes depend on full key ✔️
9. Reduces redundancy and improves structure. ⬇️
10. 2NF is used after converting table into 1NF.

◆ Step 3: Convert to BCNF

👉 Split the table 🗨

Table 1: Course

Course	Instructor
DBMS	X
AI	Y

👉 Now: Course → Instructor ✔ (Course is key)

Table 2: Student_Course

Student	Course
A	DBMS
B	DBMS
C	AI



◆ Why this works

- No dependency violates BCNF

13. Boyce-Codd Normal Form (BCNF) (10 Points)

1. BCNF – A stronger version of 3NF used to remove remaining redundancy.
2. A table is in BCNF if **for every functional dependency** ($X \rightarrow Y$), **X is a super key**
3. It ensures that only keys determine other attributes.
4. Removes hidden anomalies not solved by 3NF.
5. **Example (Not in BCNF):**

`R(Student, Course, Instructor)`

FD: $\text{Course} \rightarrow \text{Instructor}$

Here Course is not a key ❌

6. So it violates BCNF.

7. **Convert to BCNF:**

Table1: `Course(Course, Instructor)`

Table2: `Student_Course(Student, Course)`

8. Now all dependencies are based on keys ✓
9. Improves data consistency and reduces redundancy. ↓
10. BCNF is used after applying 3NF.

◆ After Conversion (3NF)

↑ Share

Table 1: Employee

ID	Dept_ID
1	D1
2	D2

Table 2: Department

Dept_ID	Dept_Name	📄
D1	Computer	
D2	Mechanical	

◆ Why this works

- Dept_Name now depends **only** on Dept_ID
- No indirect dependency
- No duplication



🌸 Simple Trick

+ Ask anything



13. Boyce-Codd Normal Form (BCNF) (10 Points)

1. BCNF – A stronger version of 3NF used to remove remaining redundancy.
2. A table is in BCNF if **for every functional dependency** ($X \rightarrow Y$), **X is a super key**
3. It ensures that only keys determine other attributes.
4. Removes hidden anomalies not solved by 3NF.
5. **Example (Not in BCNF):**

`R(Student, Course, Instructor)`

FD: $\text{Course} \rightarrow \text{Instructor}$

Here Course is not a key ❌

6. So it violates BCNF.

7. **Convert to BCNF:**

Table1: `Course(Course, Instructor)`

Table2: `Student_Course(Student, Course)`

8. Now all dependencies are based on keys ✓
9. Improves data consistency and reduces redundancy. ↓
10. BCNF is used after applying 3NF.

14. Decomposition (Lossless & Lossy) (10 Points)

1. **Decomposition** – Process of breaking a large table into smaller tables.
2. It is used to reduce redundancy and improve database design.
3. Smaller tables are related using keys.
4. **Lossless Decomposition** – No data is lost after decomposition.
5. Original table can be obtained by joining smaller tables.

6. *Example:*

$R(A,B,C) \rightarrow R1(A,B) \text{ and } R2(B,C)$ ✓ (can join back using B)

7. **Lossy Decomposition** – Some data is lost after decomposition.
8. Original table cannot be obtained correctly.

9. *Example:*

$R(A,B,C) \rightarrow R1(A,B) \text{ and } R2(A,C)$ ✗ (may produce wrong combinations)

10. Lossless decomposition is always preferred in database design.

15. Intelligent Query Processing (IQP) (10 Points)

1. IQP (Intelligent Query Processing) – Technique used to process SQL queries efficiently.
2. It helps in selecting the best way to execute a query.
3. It uses optimization methods to reduce execution time.
4. It improves overall database performance.
5. It analyzes different query execution plans.
6. Chooses the most efficient plan among them.
7. Reduces cost of data retrieval.
8. Helps in handling large amount of data efficiently.
9. Improves response time for users.
10. IQP is important for fast, efficient, and optimized query processing in DBMS.

Evolution of Query Processing Techniques (10 Points)

1. **Query Processing** – It is the process of converting SQL queries into efficient execution plans.
2. **Early Stage (Simple Processing)** – Queries were executed directly without optimization.
3. Execution was slow and inefficient for large data.
4. **Introduction of Query Optimization** – System started choosing better execution plans.
5. Used techniques like indexing and join optimization.
6. **Cost-Based Optimization** – Different plans are compared using cost (time, memory).
7. Best plan is selected based on minimum cost.
8. **Heuristic Optimization** – Uses rules to improve query performance quickly.
9. **Modern Techniques (IQP)** – Uses intelligent methods and advanced optimization.
10. Evolution improved speed, efficiency, and performance of query execution.

16. Need for Intelligent Query Optimization (10 Points)

1. **Query Optimization** – Process of selecting the best execution plan for a query.
2. It is needed to **reduce query execution time**.
3. Databases store large amount of data, so simple execution becomes slow.
4. Optimization helps in **efficient use of resources** (CPU, memory).
5. It reduces **disk I/O operations**, which improves performance.
6. Multiple ways exist to execute a query; optimization selects the best one.
7. Improves **response time** for users.
8. Helps in handling **complex queries** efficiently.
9. Essential for **modern databases and big data systems**.
10. Overall, it ensures fast, efficient, and cost-effective query processing.